Name : AKASH SAMPATH

Topic : Research Questions and Answers

Q1. Why Docker Volume ? Where data is stored ?

- Docker containers are temporary and if container is deleted the data inside container is lost
- Docker volume is used to store data permanently and the data is shared between containers
- Also data is kept safe after container is deleted and also uses the same data for new containers
- Also as containers are temporary and if you delete a container , the data inside it is lost forever
- So docker volume stores data even if the container is deleted and also allow multiple containers to share the same data
- So without volume the container has the data inside and when container is deleted the data is lost
- So with volume the container has the data inside the volume and when we delete container the data is safe

Docker volumes are stored on the host machine : Using the path :

```
/var/lib/docker/volumes/
```

We can check using the following commands :

```
docker volume ls
```

```
docker volume inspect
akashvolume
```

From our document :

```
docker volume create
akashvolume
```

```
docker volume ls
```

Q2. Why for container we should add volume ?

- We add volume because the container data is temporary and the application needs permanent storage
- Also the logs should not be lost and database data must be stored and also multiple containers share the same data
- So in MySQL container has the data base data and if container is deleted the Database data is lost
- So to get that data back or to be exist we attach volume
- So if your container crashes or you update it to new a version , your data such as database remains safe in volume
- You can attach the same volume to multiple containers so they can access the same files
- So without a volume , the data is lost when container is deleted and multiple containers cannot share data easily and backups are difficult
- So with a volume data is stored even after container is deleted and multiple containers can read or write the same data and also can backup and transfer easily

Using the following command : Attaches the volume to specific folder inside the container :

```
docker run -dt --mount
source=akashvolume,destination=/home --name akash1 nginx
```

Q3. To store data in container by volume attach the data is processed .

- This means when you attach volume , the container writes data to volume and volume stores data outside the container and new container can reuse the same data
- So if you write a file inside the containers mounted directory and its actually being written to the volume on your host machine
- So even after the container is removed , the data stays in the volume and can be picked up by a new container
- So while you attach a volume , you are creating a volume or Docker creates one and you mount it to a container path
- So any data written inside that container path is actually written to the host volume directory
- So if the container is deleted , the volume is remains and new container can mount the same volume and access the existing data

Based on the commands :	**Usage :**
`/mnt`	Inside the container's mount directory
`docker rm`	Container is removed
`docker exec -it akash1 bash`	Enter the container
`cd home | touch file1`	Create data inside the mounted volume
`docker rm akash1`	Removing the container

`/home | /mnt`	Mount it to a container path

Final Full Commands :	**Usage :**
`docker volume create akashvolume`	Creating volume
`docker run -dt --mount source=akashvolume,destination=/home --name akash1 nginx`	Running container with volume
`docker exec -it akash1 bash`	Entering the container
`cd home` `touch file1`	Creating a file
`docker stop akash1`	Stop and delete container

`docker rm akash1`	
`docker run -dt --mount source=akashvolume,destination=/mnt --name akash2 nginx`	Create new container with same volume
`cd mnt` `ls`	Checking the file created

Q4. Your application has some data to write in disk and use volume map .

- So first application writes logs and use the host volume to the container folder and the container has application's data which is stored in volume
- When you map a volume to the applications data directory inside the container like if your application writes logs to the application , You mount a volume there so logs can exists

Based on the commands : **Usage :**

`/var/log/app.log`	Application writes logs
`docker run -dt --mount source=akashvolume,destination=/home --name akash1 nginx`	Tells the container whenever the application writes to `/home` , Send that data to the `akashvolume` on my physical disk
`--mount`	App needs to write to disk you use this

Q5. Volume map can be defined in docker compose and docker run .

- So actually both support volume mapping and this focuses on the running the docker which is the command line interface way to start containers
- Using docker compose this saves files configuration so you can start complex setups with this
- Using docker run is done via the command line for quick and single containers setups
- Using docker compose this is better for complex applications with multiple containers like web app + database as it records the configuration in a file with given the version

Based on the commands :	**Usage :**
`docker run`	Starting containers
`.yml`	Saves files configuration in docker compose
`docker-compose up`	Start all complex setups with one command

Some commands :	**Usage :**
`docker run -dt --mount source=akashvolume,destination=/mnt nginx`	Running docker run
`docker compose up -d`	Running all together

`version: "3"`
 	
`services:`
`  web:`
`  image: nginx`
`  volumes:`
`  - akashvolume:/mnt`
 	
`volumes:`
`  akashvolume:`
	Using docker compose with file
	
	docker-compose.yml

Q6. What all you do in docker run can also be done in docker compose ?

Actually everything is possible in docker run and also possible in docker compose

Based on the table : docker run and docker compose :

Feature	docker run	docker compose
Run container	Possible	Possible
Port Mapping	Possible	Possible
Volume mapping	Possible	Possible
Network	Possible	Possible
Environment	Possible	Possible
Multiple Containers	Difficult & Not possible	Possible & Easy
Auto restart	Possible	Possible
Build image	Not Possible	Possible

Some commands that can do everything with docker run and docker-compose.yml (Equivalent) : Table :

docker run	docker-compose.yml
`--name`	`container_name:`
`-p 80:80`	`ports:`
`-v volume:path or --mount`	`volumes:`
`--network`	`networks:`
`-d`	`detach: true`
`--restart`	`restart:`
`--rm`	`auto_remove: true`
`-w /dir`	`working_dir:`
`--user`	`user:`

Important Commands :

Volume commands :

```
docker volume create akashvolume
```

```
docker volume ls
```

Mount volume in docker run commands :

```
docker run -dt --mount
source=akashvolume,destination=/home --name akash1 nginx
```

```
docker run -dt --mount
source=akashvolume,destination=/mnt --name akash2 nginx
```

Verifying data Deleted or Existing Commands :

```
docker exec -it akash1 bash
```

```
cd home
touch file1
exit
```

```
docker stop akash1
docker rm akash1
```

```
docker run -dt --mount source=akashvolume,destination=/mnt --name akash2 nginx
docker exec -it akash2 bash
```

```
cd mnt
ls   # you will see file1
```

Volume with network and port mapping Commands :

```
docker network create --driver=bridge --subnet=192.168.0.0/24
akashbridge
```

```
docker run -dt -p 56:80 \
  --mount source=akashvolume,destination=/mnt \
  --network=akashbridge \
  --name akash1 nginx
```

Volume in Docker file Commands : Building and running :

```
docker build -t akashdockeraws/akashdocker1008:ubuntuimage .
```

```
docker run -dt --name akash123
akashdockeraws/akashdocker1008:ubuntuimage
```